

7.4 实验四：目标识别与地理坐标定位计算

7.4.1 实验目的

本实验旨在掌握如何订阅无人机的姿态信息，以及订阅图像识别话题并提取目标的像素坐标，结合无人机的 GPS、相对高度、航向角信息，理解坐标转换，实现从图像中的目标像素坐标到实际地面目标的经纬度坐标的转换。增强数据处理能力，对多个目标图像处理得到最终的目标经纬度。将目标的经纬度提供给后续任务（如修改航点、投弹等）使用，确保系统具备精确的目标定位能力。

7.4.2 实验说明

本实验面向飞控自动任务链中的关键环节——目标识别与定位计算。学生将在本实验中学学习如何通过 ROS 框架订阅来自图像识别模块发布的 `/objects` 话题，获取图像中目标的像素位置。以及通过 MAVROS 文档订阅无人机姿态信息，学生将结合无人机实时位置、相对高度和航向角信息，基于坐标变换方法，完成将图像中的目标像素坐标精准转换为地面目标的经纬度坐标。为后续的航点修改、任务执行（投弹）奠定基础。程序包括：

（1）订阅无人机的位置信息：通过订阅无人机的位置、航向角和相对高度数据；

（2）将像素坐标转换成地理坐标：结合无人机的位置信息，通过坐标变换计算目标的经纬度；

（3）计算 3s 内拍到的目标数量，大于 20 张则得到最终的目标坐标。

7.4.3 实验步骤

步骤一:模型部署

打开 MobaXterm，在泰山派 RK3566 上部署已转换为 `.rknn` 格式的 YOLOv5s 模型，可对提供的文件进行替换，路径和命名如图 7.22 所示：

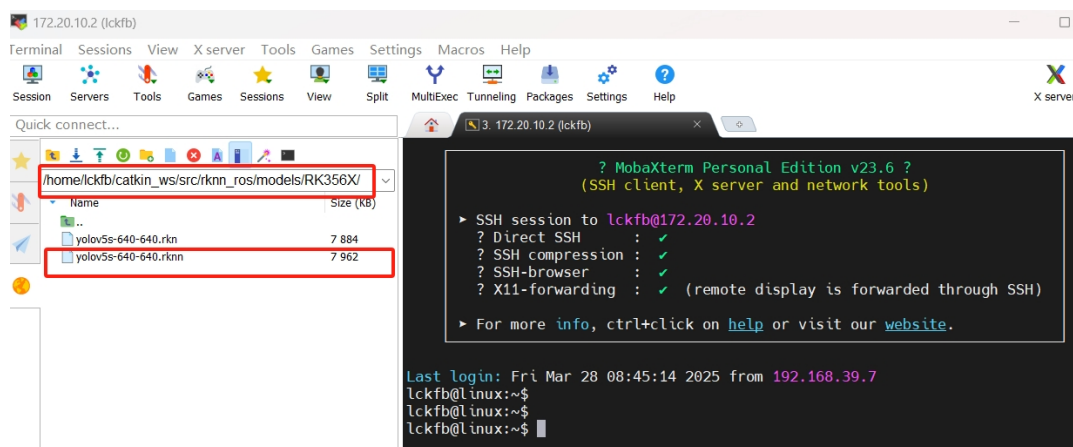


图 7.22 部署 YOLOv5s 模型

步骤二：测试模型

RK3566 连接摄像头，启动摄像头节点，命令如下：roslaunch rknn_ros camera.launch device:=video0（可通过 ls /dev/video* 查看可用的视频设备），另起一个终端，启动 yolo 识别，命令如下：roslaunch rknn_ros yolov5.launch chip_type:=RK356X，在此可实时查看到是否识别到目标以及识别的精度。可另开一个终端，输入 rqt_image_view，接收图像数据，图像化显示识别效果。可在打开的窗口选择/rknn_image/theora 显示识别后的效果。如图 7.23 所示：

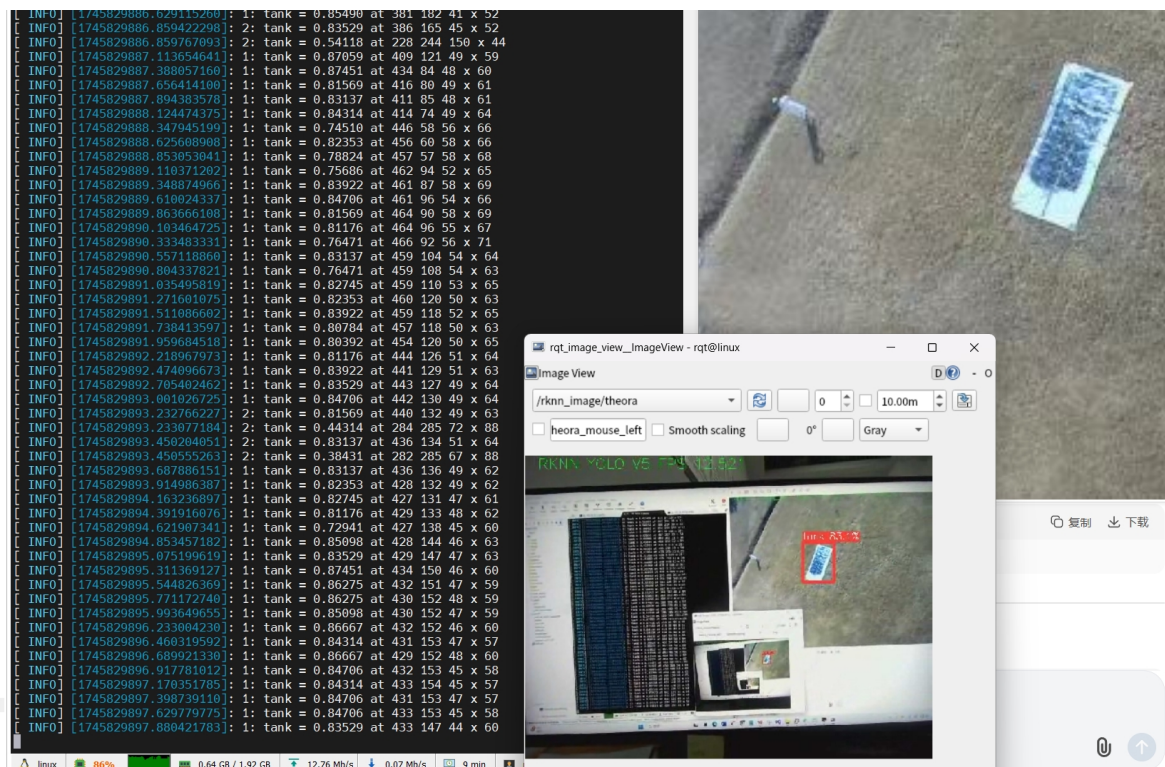


图 7.23 测试模型识别效果

步骤三：编写坐标系转换函数并计算目标地理坐标

(1) 无人机姿态相关话题：

/mavros/global_position/global（GPS,类型：sensor_msgs/NavSatFix）

/mavros/global_position/rel_alt（相对高度,类型：std_msgs/Float64）

/mavros/global_position/compass_hdg（航向角,类型：std_msgs/Float64）

在回调函数中已提取经纬度 uav_coordinates.lat 和 uav_coordinates.lon、相对高度 uav_coordinates.rel_alt、航向角 uav_coordinates.compass_hdg 等字段，存放于 uav_coordinates 结构体变量里。

(2) 像素坐标相关话题：

/objects（图像像素坐标，类型：object_information_msgs/Object）

在给出的代码里已经编写好各回调函数，结合 position_coordinates 函数计算

出了相对于 x-y 坐标系的像素坐标（xToCenter, yToCenter）。

在此基础上编写目标坐标转换函数，坐标计算函数框架：

```
//位置计算函数
void Target::position_coordinates(void){

    int xToCenter = pixel_coordinates.x - 320;      //x 方向到中心点像素
    int yToCenter = 240 - pixel_coordinates.y;      //y 方向到中心点像素
    /*****
    //坐标计算函数由学生填写

    /*****/
}
```

写入代码应实现要求：根据图像中心与像素点的相对位置关系，结合无人机的飞行高度和比例换算系数，将目标的像素坐标转换成以无人机为原点的二维直角坐标，直角坐标转换为以 y 轴为主轴的极坐标，考虑无人机的航向角信息，计算目标相对于地理坐标系正北方向的角度，转换成目标相对于无人机在地球坐标系（东-西是 X 方向，南-北是 Y 方向）的位移，再根据经纬度与实际距离的转化关系，转换成经纬度坐标增量，并加到当前无人机的经纬度坐标上，得到目标实际经纬度。坐标转换原理图如图 7.24 所示：

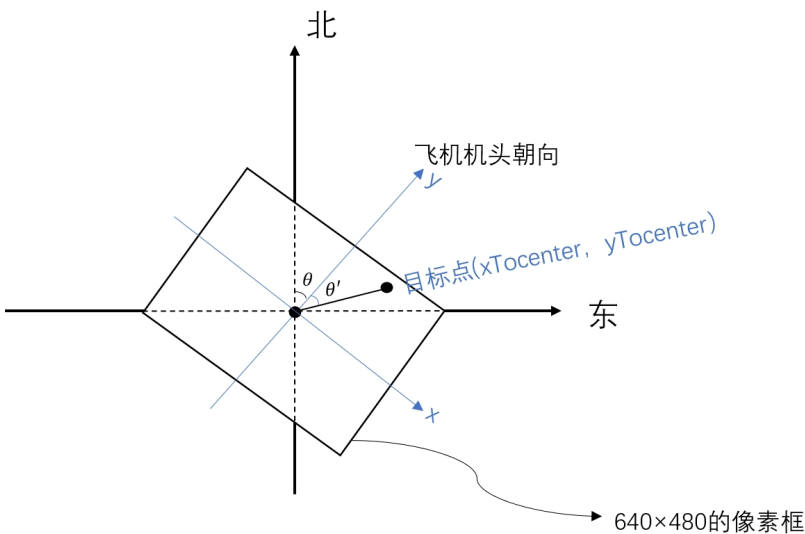


图 7.24 坐标转换原理图

求出像素距离与实际距离厘米的转换系数，说明像素距离与实际距离的转化原理如图 7.25 所示：

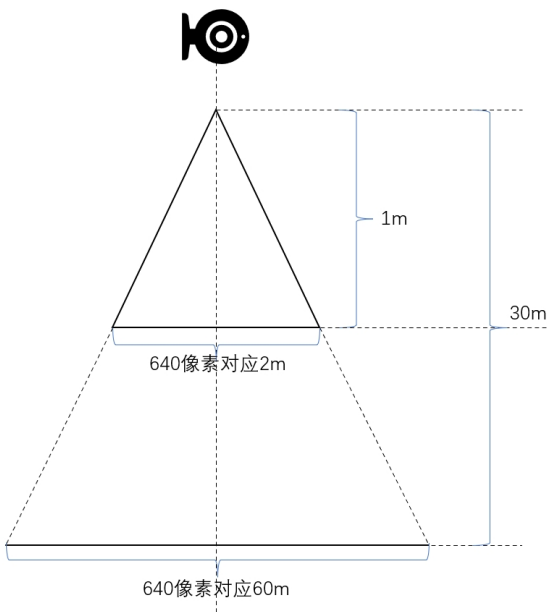


图 7.25 像素距离与实际距离的转化关系

求出经纬度与实际距离厘米的转换系数，经纬度与实际距离的转化原理示意图如图 7.26 所示：

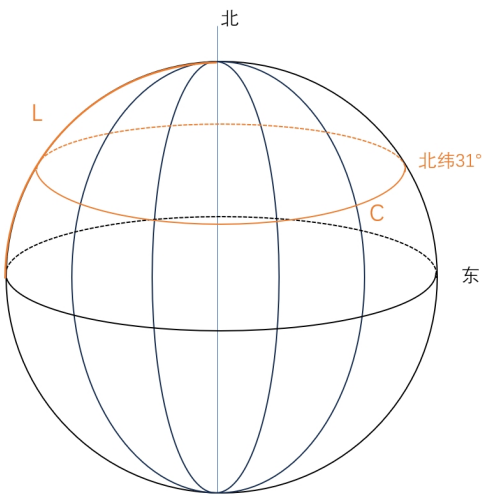


图 7.26 经纬度与实际距离的转化关系

其中 C 是地球北纬 31° 的经度周长，除以 360 就是一经度对应的实际距离，L 是地球赤道周长的 1/4,除以 90 就是一纬度对应的实际距离。从而算出经纬度与实际距离的对应关系以及像素与实际距离的对应关系。

步骤四：计算目标最终经纬度坐标

首先需要限制目标经纬度必须在预设的地理范围内，超出范围则丢弃。每次

识别到新目标时，可用结构体变量 `temp_coordinates` 暂时储存目标经纬度坐标，判断 `final_coordinates_buf` 是否为空，为空就将其加入到 `final_coordinates_buf` 缓存队列中。不为空就判断当前消息的时间是否在 `final_coordinates_buf` 中最旧的消息的 3s 内，如果在，就填充 `final_coordinates_buf`。计算 3 秒内接收到的目标数量。如果 3s 时候缓存数量超过设定阈值 20，则使用这些目标坐标计算均值作为最终的目标经纬度坐标，赋值给 `final_coordinates` 结构体变量，并存入给 `my_final_coordinates` 缓冲区，并赋值给投弹点坐标 `drop_coordinates_lat`，清空 `final_coordinates_buf` 缓冲区以便进行下一次计算。如果目标数据数量不超过 20，则清空缓冲区，进行下一次计算。计算最终目标坐标函数框架：

```
//最终目标坐标计算函数
void Target::final_target_coordinates(int32_t targetlat, int32_t targetlon){
    if(targetlat < SOUTH_lat_Scope || targetlat > NORTH_lat_SCOPE ||
targetlon < WEST_Lon_Scope || targetlon > EAST_Lon_Scope){
        return;    //超过边界阈值
    }
    Final_Coordinates temp_coordinates;
    temp_coordinates.timestamp = ros::Time::now();
    /*****
    //计算最终目标坐标函数由学生填写
    *****/

    /*****/
}
```

其中，`SOUTH_Lat_Scope` 以及其他三个相似变量都是为了限定投弹范围的，在预设航线的投弹点方圆 20m，这是为了防止偏差过大，发生危险。

步骤五：编写 `saveToFile()` 函数

可以将需要打印的数据信息放在 `saveToFile` 函数里面，设置了定时器，半分钟触发一次，指定输出文件夹路径。本实验保存路径是：
`/home/lckfb/catkin_ws/savedata/`，保存文件函数代码框架：

```
// 定时器回调函数，用于保存文件
void Target::saveToFile(const ros::TimerEvent&) {
    // 获取格式化的当前时间并创建文件名
    std::string formatted_time = formatRosTime(ros::Time::now());
    std::string folder_path = "/home/lckfb/catkin_ws/savedata/"; // 指定文件夹
    路径
    std::string filename = folder_path + formatted_time + ".txt";

    // 打开文件进行写入
    std::ofstream file(filename);
```

```

if (file.is_open()) {

    // 写入所有像素坐标数据
    for (const auto& data : pixel_coordinates_buf) {
        file << data.timestamp << " | pixel_x: " << data.x
            << ", pixel_y: " << data.y << "\n";
    }

    // 清空缓冲区
    pixel_coordinates_buf.clear();
    file.flush();
    /*****/
    //保存文件程序由学生填写

    /*****/

    // 关闭文件
    file.close();
    ROS_INFO("File written successfully. ");
} else {
    ROS_ERROR("Unable to open file for writing.");
}
}

```

三个.cpp 文件共用一个 main 函数，已在 coordinate_cal.cpp 文件里给出。把在虚拟机里写好的程序 coordinate_cal 包放进 RK3566 的 catkin_ws 文件夹下面的 src 里面，编译，运行。

参考代码：

```

// 函数用于将平面直角坐标系转换为极坐标系
void Target::convertRectToPolar(double x, double y, double &r, double &theta) {
    r = sqrt(x * x + y * y);
    // atan2 返回的是相对于 x 轴的逆时针角度，我们需要转换为相对于 y 轴的
    顺时针角度
    // 因此，我们从 90 度（或 PI/2 弧度）开始，减去 atan2 返回的角度，
    然后转换为度
    theta = 90.0 - atan2(y, x) * 180.0 / M_PI;
    // 如果 theta 是负数，我们需要加上 360 度来转换为顺时针角度
    if (theta < 0) {
        theta += 360.0;
    }
}

// 函数用于将极坐标系转换回平面直角坐标系
void Target::convertPolarToRect(double r, double theta, double &x, double &y) {
    // 将角度从度转换为弧度
    double thetaRadians = theta * M_PI / 180.0;
}

```

```

// 由于我们的极坐标系是以 y 轴为主轴，所以我们需要调整 theta
thetaRadians = M_PI / 2 - thetaRadians;
x = r * cos(thetaRadians);
y = r * sin(thetaRadians);
}
//目标位置计算函数
void Target::position_coordinates(void){
    int xToCenter = pixel_coordinates.x - 320;          //x 方向到中心点像素
    int yToCenter = 240 - pixel_coordinates.y;          //y 方向到中心点像素
    double xCentiMeters = xToCenter * pixelToCentiMeters *
(uav_coordinates.rel_alt);    //cm
    double yCentiMeters = yToCenter * pixelToCentiMeters *
(uav_coordinates.rel_alt);    //cm
    double uav_radius = 0;    //半径
    double uav_theta = 0;    //角度
    convertRectToPolar(xCentiMeters, yCentiMeters, uav_radius, uav_theta);
//直角坐标系转极坐标系
    double global_theta = uav_theta + uav_coordinates.compass_hdg;
//地球坐标系角度
    if(global_theta > 360){
        global_theta -= 360;
    }
    double xglobalCentiMeters = 0;
    double yglobalCentiMeters = 0;

convertPolarToRect(uav_radius,global_theta,xglobalCentiMeters,yglobalCentiMe
ters); //极坐标系转直角坐标系
    double targetlat = (yglobalCentiMeters / LatToCentiMeters) +
uav_coordinates.lat;
    double targetlon = (xglobalCentiMeters / LonToCentiMeters) +
uav_coordinates.lon;
    target_coordinates.timestamp = pixel_coordinates.timestamp;
    target_coordinates.lat = targetlat;
    target_coordinates.lon = targetlon;
    final_target_coordinates(targetlat,targetlon);
    ROS_INFO("Target Lat: [%.7lf], Target Lon: [%.7lf]", targetlat, targetlon);
    target_coordinates.uav_lat = uav_coordinates.lat;
    target_coordinates.uav_lon = uav_coordinates.lon;
    target_coordinates.uav_rel_alt = uav_coordinates.rel_alt;
    target_coordinates.compass_hdg = uav_coordinates.compass_hdg;
    target_coordinates.x = pixel_coordinates.x;
    target_coordinates.y = pixel_coordinates.y;
    target_coordinates.uav_timestamp = uav_coordinates.timestamp;
    target_coordinates_buf.push_back(target_coordinates);
}
//最终目标坐标计算函数
void Target::final_target_coordinates(double targetlat, double targetlon){
    if(targetlat < SOUTH_lat_Scope || targetlat > NORTH_lat_Scope || targetlon
< WEST_Lon_Scope || targetlon > EAST_Lon_Scope){

```

```

        return;
    }
    Final_Coordinates temp_coordinates;
    temp_coordinates.timestamp = ros::Time::now();
    //如果这个 buf 现在是空的，就直接填充当前内容
    if(final_coordinates_buf.empty()) {
        temp_coordinates.lat = targetlat;
        temp_coordinates.lon = targetlon;
        final_coordinates_buf.push_back(temp_coordinates);
    }else{
        if(ros::Time::now() - final_coordinates_buf.front().timestamp <
ros::Duration(3)){
            temp_coordinates.lat = targetlat;
            temp_coordinates.lon = targetlon;
            final_coordinates_buf.push_back(temp_coordinates);
        }else{
            if(final_coordinates_buf.size() > 20){
                double sum_lat = 0;
                double sum_lon = 0;
                for(auto it = final_coordinates_buf.begin(); it!=
final_coordinates_buf.end(); ++it){
                    sum_lat += it->lat;//累加纬度
                    sum_lon += it->lon;//累加经度
                }
                //最终经纬度坐标
                final_coordinates.lat = sum_lat / final_coordinates_buf.size();
                final_coordinates.lon = sum_lon /
final_coordinates_buf.size();
                final_coordinates.timestamp = temp_coordinates.timestamp;
                if(find_goal == 0){
                    drop_coordinates_lat = final_coordinates.lat;
                    drop_coordinates_lon = final_coordinates.lon;
                    find_goal = 1 ;    //发现目标标志
                }
                my_final_coordinates.push_back(final_coordinates);
                ROS_INFO("Final Target Lat: [%.7lf],Final Target Lon:
[%.7lf]", targetlat, targetlon);
                final_coordinates_buf.clear();//清空缓冲区
            }
        }else{
            final_coordinates_buf.clear();
        }
    }
}
}
}

```